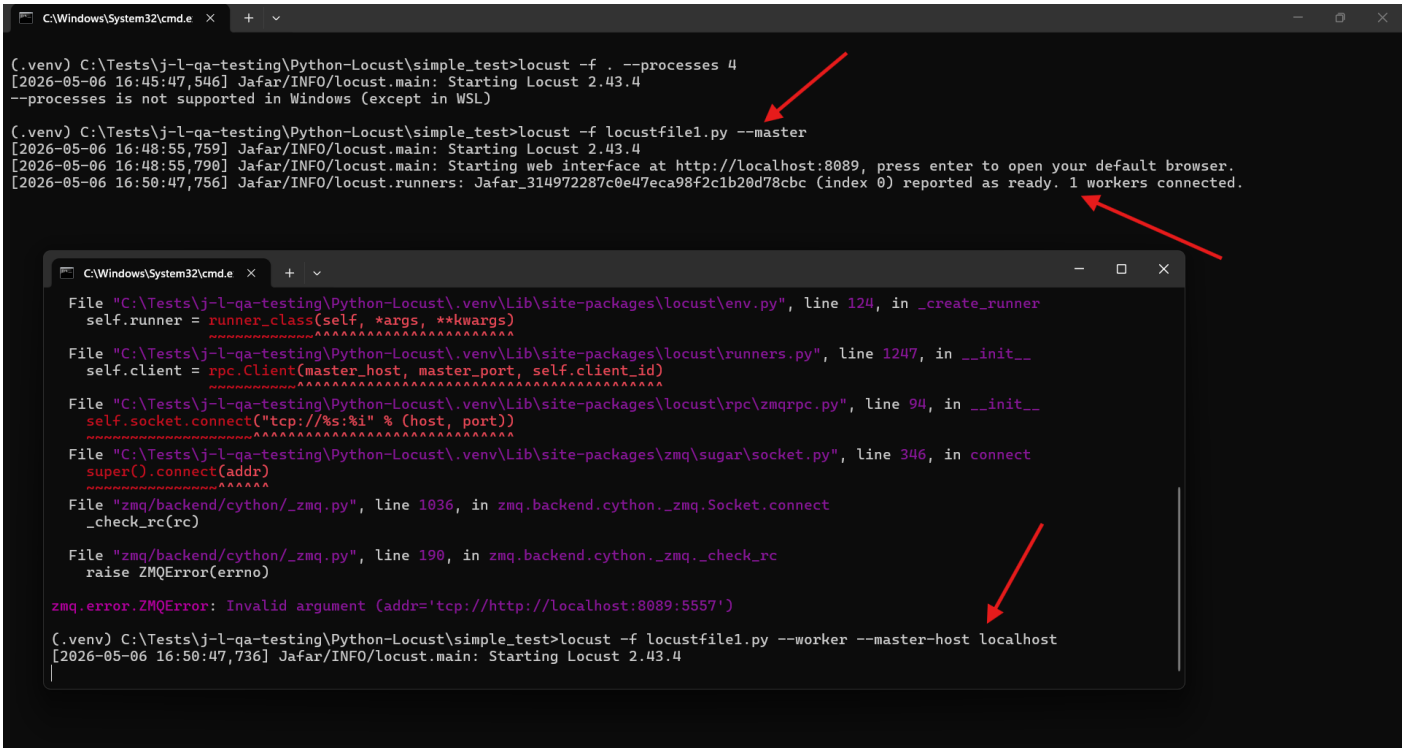


==========

Note

The `-f -` argument tells Locust to get the locustfile from master instead of from its local filesystem. This only works for single locustfiles.

==========

Options for distributed load generation

`--master-host <hostname or ip>`

Optionally used together with `--worker` to set the hostname/IP of the master node (defaults to localhost)

`--master-port <port number>`

Optionally used together with `--worker` to set the port number of the master node (defaults to 5557).

`--master-bind-host <ip>`

Optionally used together with `--master`. Determines which network interface the master node will bind to. Defaults to * (all available interfaces).

`--master-bind-port <port number>`

Optionally used together with `--master`. Determines what network ports that the master node will listen to. Defaults to 5557.

`--expect-workers <number of workers>`

Used when starting the master node with `--headless`. The master node will then wait until X worker nodes has connected before the test is started.

==========

Communicating across nodes

When running Locust in distributed mode, you may want to communicate between master and worker nodes in order to coordinate the test. This can be easily accomplished with custom messages using the built in messaging hooks:

```
from locust import events
from locust.runners import MasterRunner, WorkerRunner

# Fired when the worker receives a message of type 'test_users'
def setup_test_users(environment, msg, **kwargs):
    for user in msg.data:
        print(f"User {user['name']} received")
    environment.runner.send_message('acknowledge_users', f"Thanks for the {len(msg.data)} users!")

# Fired when the master receives a message of type 'acknowledge_users'
def on_acknowledge(msg, **kwargs):
    print(msg.data)

@events.init.add_listener
def on_locust_init(environment, **_kwargs):
    if not isinstance(environment.runner, MasterRunner):
        environment.runner.register_message('test_users', setup_test_users)
    if not isinstance(environment.runner, WorkerRunner):
        environment.runner.register_message('acknowledge_users', on_acknowledge)

@events.test_start.add_listener
def on_test_start(environment, **_kwargs):
    if not isinstance(environment.runner, WorkerRunner):
        users = [
            {"name": "User1"},
            {"name": "User2"},
            {"name": "User3"},
        ]
        environment.runner.send_message('test_users', users)
```

==========

Using the default options while registering a message handler will run the listener function in a **blocking way**, resulting in the heartbeat and other messages **being delayed for the amount of the execution**. If you think that your message handler will need to run for more than a second, then you can register it as **concurrent**. Locust will then make it run in its own greenlet. Note that these greenlets will never be joined.

==========

```
services:
    master:
        image: locustio/locust:master
        ports:
            - "8089:8089"
        volumes:
            - ./:/mnt/locust
        command: -f /mnt/locust/locustfile3.py --master -H http://master:8089

    worker:
        image: locustio/locust:master
        volumes:
            - ./:/mnt/locust
        command: -f /mnt/locust/locustfile3.py --worker --master-host master
        depends_on:
            - master
```

To start a master node and 4 workers using the following command:

docker-compose up --scale worker=4

==========

OpenTelemetry image

If you need OpenTelemetry support, use the `locustio/locust-otel` image which has the `locust[otel]` extras pre-installed:

```
docker run -p 8089:8089 -v $PWD:/mnt/locust locustio/locust-otel -f /mnt/locust/locustfile.py --otel
```

==========

Running without the web UI

You can run locust without the web UI by using the `--headless` flag together with `-u/--users` and `-r/--spawn-rate`:

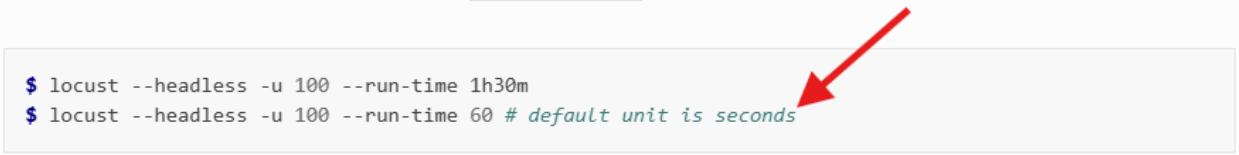
```
$ locust -f locust_files/my_locust_file.py --headless -u 100 -r 5
[2021-07-24 10:41:10,947] .../INFO/locust.main: No run time limit set, use CTRL+C to interrupt.
[2021-07-24 10:41:10,947] .../INFO/locust.main: Starting Locust 2.43.4
[2021-07-24 10:41:10,949] .../INFO/locust.runners: Ramping to 100 users using a 5.00 spawn rate
Name                 # reqs      # fails  |     Avg     Min     Max  Median  |   req/s failures/s
--------------------------------------------------------------------------------------------------
GET /hello             1     0(0.00%)  |     115     115     115     115  |    0.00    0.00
GET /world             1     0(0.00%)  |     119     119     119     119  |    0.00    0.00
--------------------------------------------------------------------------------------------------
Aggregated             2     0(0.00%)  |     117     115     119     117  |    0.00    0.00
(...)
[2021-07-24 10:44:42,484] .../INFO/locust.runners: All users spawned: {"HelloWorldUser": 100} (100 total
(...)
```

Even in headless mode you can change the user count while the test is running. Press `w` to add 1 user or `W` to add 10. Press `s` to remove 1 or `S` to remove 10.

==========

Setting a time limit for the test

To specify the run time for a test, use `-t/--run-time`:



```
$ locust --headless -u 100 --run-time 1h30m
$ locust --headless -u 100 --run-time 60 # default unit is seconds
```

Locust will shut down once the time is up. Time is calculated from the start of the test (not from when ramp up has finished).

==========

Allow tasks to finish their iteration on shutdown

By default, Locust will stop your tasks immediately (without even waiting for requests to finish). To give running tasks some time to finish their iteration, use `-s/--stop-timeout`:

```
$ locust --headless --run-time 1h30m --stop-timeout 10s
```

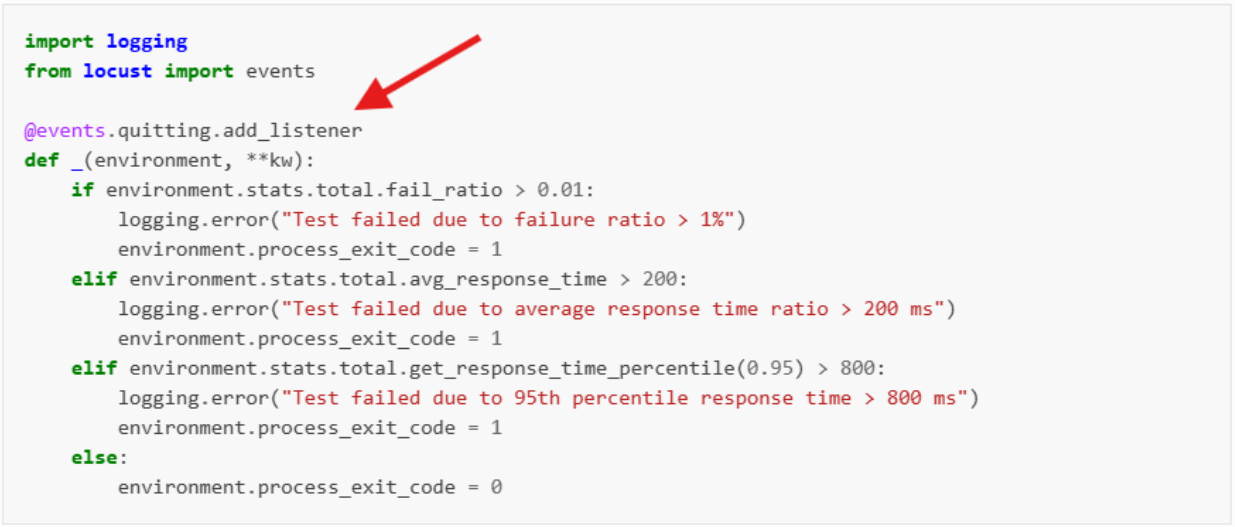
==========

Example about changing the exit code depending on conditions:

Note: 1-> we have a failure

Below is an example that'll set the exit code to non zero if any of the following conditions are met:

- More than 1% of the requests failed
- The average response time is longer than 200 ms
- The 95th percentile for response time is larger than 800 ms



```
import logging
from locust import events

@events.quitting.add_listener
def _(environment, **kw):
    if environment.stats.total.fail_ratio > 0.01:
        logging.error("Test failed due to failure ratio > 1%")
        environment.process_exit_code = 1
    elif environment.stats.total.avg_response_time > 200:
        logging.error("Test failed due to average response time ratio > 200 ms")
        environment.process_exit_code = 1
    elif environment.stats.total.get_response_time_percentile(0.95) > 800:
        logging.error("Test failed due to 95th percentile response time > 800 ms")
        environment.process_exit_code = 1
    else:
        environment.process_exit_code = 0
```

Note that this code could go into the locustfile.py or in any other file that is imported in the locustfile.

==========

Running in CI/CD

You can easily run a single instance of Locust in headless mode as part of a CI/CD pipeline.

Here's an example using GitHub Actions. Use it in combination with the above snippet to fail the run based on metrics.

```yaml
env:
    PYTHONUNBUFFERED: 1 # ensure we see output right away

jobs:
    loadtest:
        runs-on: ubuntu-latest
        timeout-minutes: 15 # just in case something goes wrong
    steps:
        - uses: actions/checkout@v6
        - uses: actions/setup-python@v6
          with:
            python-version: '3.11'
        - run: pip install locust
        - run: locust --headless --run-time 5m
```

==========

Custom load shapes

Sometimes, a completely custom-shaped load test is required that cannot be achieved by simply setting or changing the user count and spawn rate. For example, **you might want to generate a load spike** or **ramp up and down at custom times**. **By using a *LoadTestShape* class**, you have full control over the user count and spawn rate at all times.

==========

One further method may be helpful for your custom load shapes: ***get_current_user_count(),*** which returns the total number of active users. This method can be used to prevent advancing to subsequent steps **until the desired number of users has been reached**. This is especially useful if the initialization process for each user is slow or erratic in terms of how long it takes.

==========

The command with csv statistics: locust -f locustfile1.py --headless -u 10 -t25s -r 10 --csv example --csv-full-history -H http://example.com

==========

You can also customize how frequently this is written:

```
import locust.stats
locust.stats.CSV_STATS_INTERVAL_SEC = 5 # default is 1 second
```

==========

Testing other systems/protocols

Locust only comes with built-in support for HTTP/HTTPS but it can be extended to test almost any system. This is normally done by wrapping the protocol library and triggering a `request` event after each call has completed, to let Locust know what happened.

Note

It is important that the protocol libraries you use can be monkey-patched by gevent.

Almost any libraries that are pure Python (using the Python `socket` module or some other standard library function like `subprocess`) should work fine out of the box - but if they do their I/O calls from compiled code C, gevent will be unable to patch it. This will block the whole Locust/Python process (in practice limiting you to running a single User per worker process).

Some C libraries allow for other workarounds. For example, if you want to use psycopg2 to performance test PostgreSQL, you can use psycogreen. If you are willing to get your hands dirty, you may be able to patch a library yourself, but that is beyond the scope of this documentation.

==========

```python
from geventhttpclient.client import HTTPClientPool

class MyUser(FastHttpUser):
    client_pool = HTTPClientPool(concurrency=5)
```

concurrency= *10*

Parameter passed to FastHttpSession. Describes number of concurrent requests allowed by the FastHttpSession. Default 10. Note that setting this value has no effect when custom client_pool was given, and you need to spawn a your own gevent pool to use it (as Users only have one greenlet). See test_fasthttp.py / test_client_pool_concurrency for an example.

connection_timeout= *60.0*

Parameter passed to FastHttpSession

insecure= *True*

Parameter passed to FastHttpSession. Default True, meaning no SSL verification.

max_redirects= *30*

Parameter passed to FastHttpSession.

max_retries= *0*

Parameter passed to FastHttpSession.

network_timeout= *60.0*

Parameter passed to FastHttpSession

proxy_host= *None*

Parameter passed to FastHttpSession

proxy_port= *None*

Parameter passed to FastHttpSession

==========

rest(*method*, *url*, *headers=None*, ***kwargs*) [source]

A wrapper for self.client.request that:

- Parses the JSON response to a dict called `js` in the response object. Marks the request as failed if the response was not valid JSON.
- Defaults `Content-Type` and `Accept` headers to `application/json`
- Sets `catch_response=True` (so always use a with-block)
- Catches any unhandled exceptions thrown inside your with-block, marking the sample as failed (instead of exiting the task immediately without even firing the request event)

rest_(*method*, *url*, *name=None*, ***kwargs*) [source]

Some REST api:s use a timestamp as part of their query string (mainly to break through caches). This is a convenience method for that, appending a _=<timestamp> parameter automatically